

Từ nhập môn đến thành thạo: thuật toán “zkw” cho luồng chi phí nhỏ nhất

zkw

7 tháng 7, 2011

Nguồn gốc: From beginner to proficient: the “zkw algorithm” for minimum-cost flow

Graph Theory / Minimum-cost flow algorithm - Zhang Kunwei.pdf

1 Một số khái niệm cơ bản về luồng mạng

Nhiều bạn đã từng dựng mô hình luồng mạng để giải bài, cũng đã học nhiều thuật toán khác nhau, nhưng lại không nói rõ được các khái niệm cơ bản. Tuy trong các mô hình khác nhau, tên gọi cụ thể có thể không giống nhau, tư tưởng tương ứng phía sau các tên gọi ấy vẫn nhất quán. Phần thảo luận dưới đây cố gắng chuẩn hóa cách nói; một vài chỗ cần làm rõ lại cách gọi thường dùng.

Tìm luồng khả thi. Cho một mạng luồng, ban đầu mỗi đỉnh không nhất thiết cân bằng, nghĩa là mỗi đỉnh có thể dư hoặc thiếu. Mỗi cạnh có thể có cận dưới và cận trên cho lưu lượng, và lưu lượng hiện tại trên mỗi cạnh có thể chưa thỏa các ràng buộc đó. Trong bài toán luồng khả thi không có khái niệm nguồn và đích. Mục tiêu của thuật toán là tìm một phương án làm cho tất cả các đỉnh cân bằng, mọi cạnh thỏa ràng buộc lưu lượng, đồng thời có thể kèm điều kiện chi phí nhỏ nhất, tức **luồng khả thi chi phí nhỏ nhất**.

Tìm luồng cực đại. Cho một mạng luồng có hai đỉnh đặc biệt gọi là nguồn và đích. Ngoài nguồn và đích, mỗi đỉnh đã cho đều phải cân bằng. Nguồn có thể sinh ra lượng luồng vô hạn, còn đích có thể hấp thụ lượng luồng vô hạn. Trong mô hình luồng cực đại chuẩn, nghiệm ban đầu nhất định là khả thi, chẳng hạn mọi cạnh đều có lưu lượng bằng 0, nên cạnh không được có cận dưới. Mục tiêu của thuật toán là tìm một phương án có lượng luồng từ nguồn đến đích lớn nhất mà không phá vỡ ràng buộc khả thi; đồng thời có thể kèm điều kiện chi phí nhỏ nhất, tức **luồng cực đại chi phí nhỏ nhất**.

Luồng cực đại mở rộng. Đây là bài toán tìm luồng cực đại trên mạng có cận dưới/cận trên hoặc có đỉnh dư luồng. Thực chất nó gồm hai phần: trước hết khử cận dưới, khử phần dư, và trong bài toán luồng chi phí nhỏ nhất có thể còn cần khử các lưu lượng chưa thỏa điều kiện tối ưu để tìm một luồng khả thi; sau đó mới tiếp tục tìm luồng cực đại. Vì vậy phép chuyển đổi ở đây dường như đưa luồng cực đại về luồng khả thi rồi lại quay về luồng cực đại, nhưng bản chất là biến một quá trình, tức luồng cực đại mở rộng, thành hai quá trình: luồng khả thi cộng với luồng cực đại.

Các khái niệm trên cũng áp dụng cho luồng chi phí nhỏ nhất.

2 Các phép chuyển đổi trong luồng chi phí nhỏ nhất

Không ít bạn cho rằng thuật toán khử chu trình dùng được rộng hơn thuật toán đường tăng, vì có thể có cạnh âm, chu trình âm, mỗi đỉnh đều có thể dư hoặc thiếu, v.v. Thực ra thuật toán đường tăng cũng có

thể xử lý cạnh âm, chu trình âm, cạnh dưới/cạnh trên, v.v., và nói chung còn nhanh hơn thuật toán khử chu trình. Trong phần dưới, e là lượng dư, c là chi phí, và u là dung lượng.

1. Luồng khả thi chi phí nhỏ nhất $\rightarrow$ luồng cực đại chi phí nhỏ nhất

Tạo siêu nguồn s' và siêu đích t' . Với mỗi đỉnh i , nếu $e_i > 0$ thì thêm cạnh

$$s' \rightarrow i, \quad c = 0, \quad u = e_i;$$

nếu $e_i < 0$ thì thêm cạnh

$$i \rightarrow t', \quad c = 0, \quad u = -e_i.$$

Sau đó tìm luồng cực đại chi phí nhỏ nhất từ s' đến t' . Nếu lượng luồng bằng

$$\sum_{e_i > 0} e_i,$$

thì tồn tại luồng khả thi; mạng dư khi ấy đã chứa lời giải trên đồ thị ban đầu.

2. Luồng cực đại chi phí nhỏ nhất $\rightarrow$ luồng khả thi chi phí nhỏ nhất

Thêm cạnh

$$t \rightarrow s, \quad c = -\infty, \quad u = +\infty,$$

đặt $e_i = 0$ cho mọi đỉnh i , rồi trực tiếp giải bài toán luồng khả thi chi phí nhỏ nhất.

3. Khử cạnh âm trong luồng khả thi chi phí nhỏ nhất

Trực tiếp cho cạnh âm chảy đầy dung lượng.

4. Khử cạnh âm trong luồng cực đại chi phí nhỏ nhất

Trước hết thêm cạnh

$$t \rightarrow s, \quad c = 0, \quad u = +\infty.$$

Dùng phương pháp ở mục 3 để khử cạnh âm, rồi dùng phương pháp ở mục 1 để tìm một luồng khả thi chi phí nhỏ nhất. Sau đó giữ nguyên nhãn khoảng cách và tiếp tục tìm luồng cực đại chi phí nhỏ nhất. Chú ý lúc này chi phí tăng thêm không thể máy móc dùng nhãn của nguồn; nó phải là hiệu giữa nhãn của nguồn và nhãn của đích.

3 Cạnh âm và chu trình âm trong luồng chi phí

Khi giải luồng chi phí, khó tránh khỏi vấn đề cạnh âm và chu trình âm. Với thuật toán khử chu trình, chu trình âm chính là một phần của thuật toán; nhưng với thuật toán đường tăng, chu trình âm là điều ta không muốn thấy. Vì trong thi đấu, dùng thuật toán khử chu trình gây vấn đề hiệu năng nghiêm trọng, ta phải tìm cách xử lý cạnh âm và chu trình âm bằng thuật toán đường tăng.

Với cạnh âm đơn thuần, nếu các cạnh âm này không tạo thành chu trình âm, ta có thể dùng kỹ thuật gán lại trọng số. Cụ thể, chọn nhãn đỉnh thích hợp để *reduced cost* không âm. Nhãn đỉnh này thường có thể lấy là khoảng cách đến đích. Khi đó trọng số cạnh được đổi thành

$$c_{ij}^\pi = c_{ij} - D_i + D_j.$$

Theo điều kiện cân bằng luồng, từ chi phí mới này có thể dễ dàng suy ra chi phí gốc; đồng thời có thể chứng minh phép gán lại trọng số như vậy không làm thay đổi nghiệm tối ưu. Cái giá phải trả là một lần chạy SPFA, tương đối nhỏ.

Nếu tồn tại chu trình âm, SPFA sẽ không dừng. Nhiều bạn thêm điều kiện giới hạn nhân tạo để bắt nó dừng; cách đó sai. Lấy một ví dụ: nguồn và đích đều là đỉnh cô lập, còn trong đồ thị có một chu trình chi phí âm, dung lượng dương. Dù khởi tạo nhãn thế nào, chỉ dựa vào tăng luồng sẽ không thể khử chu trình âm này, nên không thể nhận được nghiệm tối ưu. Theo định nghĩa luồng cực đại chi phí nhỏ nhất, ta phải tìm phương án có chi phí nhỏ nhất trong tất cả các luồng cực đại, mà ở ví dụ này mọi luồng cực đại đều có lượng luồng bằng 0. Có thể bạn sẽ nói ví dụ này quá cực đoan, nhưng cũng rất dễ dựng các ví dụ khác để thấy rằng không xử lý chu trình âm, hoặc chỉ vá thuật toán một cách đơn giản, không giải quyết được vấn đề bản chất.

Cách giải quyết là ép mọi cạnh có chi phí âm chảy đầy, gọi là thao tác “đẩy ngược luồng”. Làm vậy phá vỡ điều kiện cân bằng, nhưng thỏa điều kiện tối ưu. Sau đó ta có thể tìm luồng khả thi trước, rồi tìm luồng cực đại, trong suốt quá trình luôn duy trì điều kiện tối ưu và từng bước giải quyết điều kiện cân bằng, điều kiện luồng cực đại, v.v. Đây chính là phương pháp đã nhắc ở mục 2. Phương pháp này có thể khử mọi cạnh âm theo nghĩa reduced cost, đồng thời xử lý đúng mọi chu trình âm. Vậy tốc độ của phương pháp này ra sao?

Các đồ thị liên quan đến luồng chi phí đại khái chia làm hai loại. Loại thứ nhất thiên về chi phí, tức tổng lượng luồng không lớn, chẳng hạn *Two Shortest* chỉ có lượng luồng 2, hoặc *KaKa's Matrix Travels*. Phương pháp ép chảy đầy tổng quát hoạt động không tốt trên loại đồ thị này, vì lượng luồng ban đầu nhỏ mà cạnh âm lại nhiều; sau khi chuyển đổi, lượng luồng trong giai đoạn tìm luồng khả thi lớn hơn nhiều so với lượng luồng gốc, ảnh hưởng nặng đến tốc độ. Mặt khác, loại đồ thị này thường có bối cảnh đường đi ngắn nhất khá sâu, nên không xuất hiện chu trình âm và có thể dùng SPFA để gán lại trọng số. Loại thứ hai thiên về lưu lượng, tức chi phí của các cạnh chênh lệch không lớn, ví dụ bài *employee* hoặc bài tối ưu hóa hình ảnh. Trong loại đồ thị này, lượng luồng không nhỏ còn số cạnh âm tương đối hạn chế; số lượng cạnh âm có thể hơi lớn, nhưng điểm mấu chốt là lượng luồng phía trước lớn. Lượng luồng tăng thêm sau chuyển đổi có thể được tính rất nhanh, nên phương pháp ép chảy đầy không có vấn đề.

Ngoài ra, cách dựng đồ thị khác nhau có thể tạo ra các mô hình khác nhau. Trong bài *employee*, nếu khi dựng đồ thị ta nối cạnh từ mỗi điểm về phía sau và chạy luồng cực đại chi phí nhỏ nhất từ trước ra sau, thuật toán này chỉ có cạnh âm, không có chu trình âm. Nếu nối cạnh từ mỗi điểm về phía trước và tìm luồng cân bằng chi phí nhỏ nhất trên toàn đồ thị, chu trình âm sẽ xuất hiện. Nhưng ở đây không có khác biệt bản chất: ép mọi cạnh nối về phía trước trong mô hình thứ hai chảy đầy sẽ thu được mô hình thứ nhất; nếu ép cả các cạnh âm khác chảy đầy, ta còn thu được một mô hình hoàn toàn không có cạnh âm. Có lẽ các bạn cũng đoán được, tốc độ của các mô hình này không khác xa nhau, vì theo thảo luận vừa rồi, lượng luồng tăng thêm không chiếm vai trò chủ yếu trong quá trình chạy thuật toán.

4 “Thuật toán zkw” cho luồng chi phí nhỏ nhất

Luồng chi phí nhỏ nhất có thể xem là một nội dung tương đối ít gặp trong thi đấu OI, nhưng sự xuất hiện bất ngờ của bài *employee* ở NOI 2008 thật sự khiến rất nhiều người, kể cả chính zkw, trở tay không kịp. zkw đáng thương khi ấy đã nghĩ ra mô hình luồng chi phí nhỏ nhất, nhưng chưa từng cài đặt nó bao giờ nên không dám viết, và bài đó được 0 điểm. Hiện tại kinh nghiệm của zkw về luồng chi phí là: tuy lý thuyết khó, viết một chương trình luồng chi phí đủ AC bài lại khá đơn giản. Trước hết là một chương trình `costflow` tôi viết: chưa đến 70 dòng, luồng chi phí còn dễ viết hơn luồng cực đại.

```
#include <cstdio>
#include <cstring>
using namespace std;
```

```

const int maxint=~0U>>1;

int n,m,pil,cost=0;
bool v[550];
struct etype
{
    int t,c,u;
    etype *next,*pair;
    etype(){}
    etype(int t_,int c_,int u_,etype* next_):
        t(t_),c(c_),u(u_),next(next_){}
    void* operator new(unsigned,void* p){return p;}
} *e[550];

int aug(int no,int m)
{
    if(no==n)return cost+=pil*m,m;
    v[no]=true;
    int l=m;
    for(etype *i=e[no];i;i=i->next)
        if(i->u && !i->c && !v[i->t])
        {
            int d=aug(i->t,l<i->u?l:i->u);
            i->u-=d,i->pair->u+=d,l-=d;
            if(!l)return m;
        }
    return m-l;
}

bool modlabel()
{
    int d=maxint;
    for(int i=1;i<=n;++i)if(v[i])
        for(etype *j=e[i];j;j=j->next)
            if(j->u && !v[j->t] && j->c<d)d=j->c;
    if(d==maxint)return false;
    for(int i=1;i<=n;++i)if(v[i])
        for(etype *j=e[i];j;j=j->next)
            j->c-=d,j->pair->c+=d;
    pil += d;
    return true;
}

int main()
{
    freopen("costflow.in","r",stdin);
    freopen("costflow.out","w",stdout);
    scanf("%d %d",&n,&m);
    etype *Pe=new etype[m+m];
    while(m--)
    {
        int s,t,c,u;
        scanf("%d%d%d%d",&s,&t,&u,&c);
        e[s]=new(Pe++)etype(t, c,u,e[s]);
        e[t]=new(Pe++)etype(s, -c,0,e[t]);
        e[s]->pair=e[t];
        e[t]->pair=e[s];
    }
    do do memset(v,0,sizeof(v));

```

```

while(aug(1,maxint));
while(modlabel());
printf("%d\n",cost);
return 0;
}

```

Ở đây dùng thuật toán đường đi ngắn nhất liên tiếp. Thuật toán đường đi ngắn nhất ư? Tại sao trong chương trình không có SPFA hay Dijkstra? Khoan đã, trước hết hãy ôn lại nhãn khoảng cách trong thuật toán đường đi ngắn nhất của lý thuyết đồ thị. Định nghĩa D_i là nhãn khoảng cách của đỉnh i . Bất kỳ thuật toán đường đi ngắn nhất nào cũng bảo đảm rằng khi thuật toán kết thúc, với mọi cạnh từ đỉnh i đi ra đỉnh j ta có

$$D_i \leq D_j + c_{ij}$$

(điều kiện 1), và với mỗi i tồn tại một j làm cho dấu bằng xảy ra (điều kiện 2). Nói cách khác, bất kỳ thuật toán nào thỏa hai điều kiện trên đều có thể gọi là thuật toán đường đi ngắn nhất, chứ không chỉ SPFA hay Dijkstra. Khi thuật toán kết thúc, đúng các cạnh nằm trên đường đi ngắn nhất thỏa

$$D_i = D_j + c_{ij}.$$

Trong tính toán luồng chi phí nhỏ nhất, mỗi lần ta tăng luồng theo đường thỏa $D_i = D_j + c_{ij}$ thì không phá vỡ điều kiện 1, nhưng có thể phá vỡ điều kiện 2. Hậu quả của việc không thỏa điều kiện 2 là gì? Nó khiến ta không tìm được đường tăng mới mà mọi cạnh đều thỏa $D_i = D_j + c_{ij}$. Vì thế sau mỗi lần tăng luồng, ta thường phải dùng Dijkstra, SPFA, v.v. để tính lại nhãn khoảng cách mới thỏa điều kiện 2. Điều này hiển nhiên là lãng phí. Trong thuật toán KM, ta có thể liên tục sửa nhãn đỉnh khả thi và mở rộng đồ thị con khả thi; ở đây cũng tương tự, ta có thể liên tục sửa các nhãn khoảng cách vốn luôn thỏa điều kiện 1, cho đến khi có thể tăng luồng tiếp, tức thỏa điều kiện 2.

Hãy nhớ lại cách thuật toán KM sửa nhãn đỉnh. Dựa trên lần DFS cuối cùng không tìm được đường xen kẽ, ta tìm

$$d = \min_{i \in V, j \notin V} \{-w_{ij} + A_i + B_j\},$$

rồi tăng d cho các đỉnh bên trái và giảm d cho các đỉnh bên phải. Ở đây cũng vậy: dựa trên lần DFS cuối cùng không tìm được đường tăng, ta tìm

$$d = \min_{\substack{i \in V, j \notin V \\ u_{ij} > 0}} \{c_{ij} - D_i + D_j\},$$

rồi tăng nhãn khoảng cách của mọi đỉnh đã thăm thêm d . Có thể chứng minh cách này không phá vỡ tính chất 1, đồng thời ít nhất một cạnh mới đi vào đồ thị con thỏa $D_i = D_j + c_{ij}$.

Các bước thuật toán là: đặt nhãn ban đầu bằng 0; liên tục tăng luồng; nếu không tăng được thì sửa nhãn rồi tiếp tục tăng; dừng khi thật sự không thể tăng nữa, tức nhãn của nguồn đã được cộng đến $+\infty$. Chú ý: trong chương trình, mọi cost đều biểu diễn reduced cost, tức

$$c_{ij}^\pi = c_{ij} - D_i + D_j.$$

Ngoài ra, thuật toán này không thể dùng trực tiếp cho đồ thị có bất kỳ cạnh âm nào, càng không thể dùng trực tiếp khi có chu trình âm. Cách xử lý hai trường hợp này đã được nói ở mục 2 và mục 3.

Như vậy ta nhận được một thuật toán đơn giản, chỉ cần tăng luồng và sửa nhãn, mỗi phần chỉ có khoảng 7 dòng, không cần BFS, hàng đợi hay SPFA, nên độ phức tạp lập trình rất thấp. Vì các lần tăng luồng thực tế đều đi theo đường ngắn nhất, độ phức tạp thời gian về mặt lý thuyết giống thuật toán đường đi ngắn nhất liên tiếp dùng SPFA, v.v., nhưng tiết kiệm thời gian chạy SPFA hoặc Dijkstra. Thử nghiệm thực tế cho thấy hằng số của thuật toán này rất nhỏ, tốc độ khá nhanh; mọi dữ liệu của bài *employee* cộng lại đều chạy trong vòng 2 giây.

5 Thuật toán luồng chi phí “zkw” chậm trên những đồ thị nào

Trong thực tế, thuật toán ở trên rất lạ. Trên một số đồ thị nó chạy rất nhanh, nhưng trên những đồ thị khác lại chậm hơn thuật toán tăng luồng thuần SPFA. Không ít bạn sau khi đo thực tế đã tổng kết rằng nó nhanh hơn trên đồ thị dày và chậm hơn trên đồ thị thưa, nhưng không hoàn toàn như vậy. Ở đây tôi phân tích từ góc độ lý thuyết xem thuật toán này lý tưởng trên những loại đồ thị nào.

Trước hết phân tích quá trình tăng luồng. So với thuật toán SPFA trực tiếp, vì cả hai đều tăng luồng theo đường ngắn nhất, thao tác tăng luồng thực tế không khác nhau quá nhiều, và mỗi đường tăng cũng gần giống nhau. Vì vậy nếu không xét tăng nhiều đường cùng lúc, số lần tăng luồng về cơ bản nên như nhau. Khác biệt chính về thời gian chạy nằm ở phần tìm đường tăng.

Vậy ưu thế của thuật toán zkw nằm ở đâu? So với SPFA, cách gán lại nhãn kiểu KM rõ ràng có lợi về tốc độ: mỗi lần chỉ là một lần quét các cạnh. Còn SPFA phải duy trì nhãn và hàng đợi phức tạp hơn; đồng thời để sửa nhãn, nó cần thăm một số đỉnh nhiều hơn một lần, nên chậm hơn không ít. Ngoài ra, trong thuật toán zkw, việc tăng luồng là tăng nhiều đường, và có thể thực hiện nhiều lần tăng sau một lần gán lại nhãn. Đặc điểm này phát huy tác dụng khi nhiều đường có cùng chi phí, tiếp tục giảm thời gian tiêu tốn cho việc gán lại nhãn.

Tiếp theo hãy nghĩ về nhược điểm của thuật toán zkw, cũng chính là vấn đề của cách gán lại nhãn KM. Vấn đề chính của gán lại nhãn KM là nó không bảo đảm sau một lần gán lại nhãn sẽ tồn tại đường tăng. Trong trường hợp xấu nhất, mỗi lần chỉ thêm được một cạnh vào mạng trọng số 0. Khi đó thuật toán sẽ liên tục gán lại nhãn, liên tục thử tăng luồng nhưng lần nào cũng không tăng được, rơi vào tình trạng lợi bất cập hại.

Có lẽ các bạn đã đoán được điều tiếp theo. Với đồ thị có lượng luồng cuối cùng lớn nhưng miền giá trị chi phí không rộng, hoặc đồ thị có đường tăng ngắn như đồ thị hai phía, thuật toán zkw đều tương đối nhanh. Lý do là các ưu thế của nó được phát huy đầy đủ. Lượng luồng lớn nghĩa là có thể tăng nhiều lần với cùng một chi phí; miền chi phí hẹp nghĩa là không cần sửa nhãn khoảng cách quá nhiều đã có đường tăng mới; đường tăng ngắn cải thiện đáng kể trường hợp xấu nhất, vì ngay cả khi mỗi lần chỉ thêm một cạnh, ta cũng nhanh chóng ghép được một đường ngắn nhất. Nếu ngược lại, lượng luồng không lớn, chi phí không nhỏ, đường tăng lại dài, thì không thích hợp dùng thuật toán zkw.

Nếu không thích hợp thì làm thế nào? Hãy xem tiếp.

6 Thuật toán nguyên thủy-đối ngẫu cho luồng chi phí nhỏ nhất

Thuật toán SPFA trực tiếp cho luồng chi phí nhỏ nhất và thuật toán gán lại nhãn kiểu KM đã nói ở trên đều có những trường hợp rất chậm. Phần này viết về một thuật toán gọi là “mới”, nhưng thật ra rất kinh điển, kết hợp ưu thế của cả hai.

Trước hết nhìn vào bản chất của thuật toán zkw. Bạn đọc kỹ sẽ nhận ra quá trình chính của thuật toán là liên tục luân phiên tính đường đi ngắn nhất và luồng cực đại. Cụ thể, mỗi quá trình tăng luồng là tìm một luồng cực đại để tăng trong mạng trọng số 0, còn quá trình gán lại nhãn là tìm một bộ nhãn khoảng cách khả thi mới. Đã như vậy, luồng cực đại và đường đi ngắn nhất hoàn toàn có thể được thay bằng các phương pháp khác. Thuật toán luân phiên tính đường đi ngắn nhất và luồng cực đại để tìm luồng cực đại chi phí nhỏ nhất này thực ra chính là thuật toán **nguyên thủy-đối ngẫu** rất kinh điển.

Các thuật toán luồng chi phí đại khái chia thành hai loại. Một loại là lời giải kinh điển, như khử chu trình, đường tăng, nguyên thủy-đối ngẫu, v.v.; đặc điểm của chúng là đi từng bước chắc chắn, duy trì một trong hai tính chất khả thi hoặc tối ưu, rồi liên tục cải thiện phía còn lại. Loại kia hiện đại hơn, ví dụ điển hình là thuật toán nổi lỏng và đơn hình mạng. Do nổi lỏng ràng buộc đối với nghiệm trong quá trình giải, chúng nhanh hơn lời giải kinh điển rất nhiều, nhưng đồng thời cũng tăng độ khó lập trình và rào cản hiểu

thuật toán. Thuật toán nguyên thủy-đối ngẫu nói dưới đây đương nhiên không thể nhanh hơn nói lỏng và đơn hình mạng, nhưng có thể xem là một trong các thuật toán tốt nhất của nhóm kinh điển.

Trước hết xét lựa chọn thuật toán đường đi ngắn nhất. Trên đồ thị có lượng luồng lớn, chi phí nhỏ và khoảng cách ngắn, gán lại nhãn KM có hiệu quả tốt; nhưng trên các đồ thị khác, SPFA lại có ưu thế.

Tiếp theo xét lựa chọn thuật toán luồng cực đại. Trong cách tăng luồng bạo lực SPFA thô sơ nhất, ta hoàn toàn không duy trì nhãn khoảng cách; điều này ngầm dùng tăng một đường. Trong thuật toán luồng chi phí zkw, ta dùng tăng nhiều đường. Có cần dùng thuật toán luồng mạng cao cấp hơn không? Qua thử nghiệm thực tế, trừ khi đồ thị đặc biệt dày, hiệu quả thường không tốt. Nguyên nhân là lượng luồng có thể tăng không lớn; nếu duy trì nhãn đỉnh của SAP thì lại tăng khá nhiều chi phí phụ, và đẩy tiền luồng cũng tương tự.

Tóm lại, lựa chọn trung hòa là dùng SPFA có tối ưu *Small Label First* để duy trì nhãn khoảng cách trong thuật toán zkw, đồng thời giữ lại tăng nhiều đường. Chú ý rằng ngoài lần đầu, SPFA đều làm việc trên một đồ thị mà mọi cạnh đều có trọng số dương theo nghĩa reduced cost. Đây là điểm khác với SPFA trực tiếp không duy trì nhãn đỉnh, và cũng là tinh thần của thuật toán zkw ban đầu. Chính vì vậy, SPFA ở đây thậm chí có thể được thay bằng Dijkstra.

Thuật toán nguyên thủy-đối ngẫu đặc biệt này trên đồ thị dày, hai phía, chi phí nhỏ thì không địch lại thuật toán zkw ban đầu, nhưng vượt xa SPFA bạo lực. Trên các đồ thị khác, nó thắng ổn định cả hai. Nó nhanh hơn SPFA bạo lực vì tăng nhiều đường, đồng thời dùng reduced cost để thu hẹp miền chi phí, có lợi cho hoạt động của SPFA do số lần nói lỏng cần thiết giảm đi. Hơn nữa, sau khi dùng reduced cost thì không còn cạnh âm, khiến tối ưu SLF phát huy tác dụng thực sự. Nhớ rằng tối ưu SLF chỉ đạt hiệu quả tốt nhất khi mọi cạnh đều dương; thậm chí ta còn có thể dùng Dijkstra cho các bước sau. Nó nhanh hơn thuật toán zkw vì trên đồ thị có lượng luồng nhỏ, chi phí lớn, khoảng cách dài, sửa nhãn khoảng cách một lần cho đúng thường hiệu quả hơn điều chỉnh lặp đi lặp lại.

Cuối cùng là mã nguồn. Ví dụ lần này là POJ 3680, zkw Accepted, 216K, 360MS. Ngoài ra còn có POJ 3422, một ví dụ mà thuật toán zkw ban đầu không giải; phương pháp này chỉ dùng 47MS. Mã nguồn gần như tương tự nên không đặt ở đây.

```
#include <cstdio>
#include <cstring>
#include <deque>
#include <algorithm>
using namespace std;

const int V=440, E=V*2, maxint=0x3F3F3F3F;

struct etype
{
    int t, c, u;
    etype *next, *pair;
    etype() {}
    etype(int T, int C, int U, etype* N): t(T), c(C), u(U), next(N) {}
    void* operator new(unsigned, void* p){return p;}
} *e[V], Te[E+E], *Pe;

int S, T, n, piS, cost;
bool v[V];

void addedge(int s, int t, int c, int u)
{
    e[s] = new(Pe++) etype(t, +c, u, e[s]);
    e[t] = new(Pe++) etype(s, -c, 0, e[t]);
    e[s]->pair = e[t];
    e[t]->pair = e[s];
}
```

```

}

int aug(int no, int m)
{
    if (no == T) return cost += piS * m, m;
    v[no] = true;
    int l = m;
    for (etype *i = e[no]; i; i = i->next)
        if (i->u && !i->c && !v[i->t])
        {
            int d = aug(i->t, l < i->u ? l : i->u);
            i->u -= d, i->pair->u += d, l -= d;
            if (!l) return m;
        }
    return m - l;
}

bool modlabel()
{
    static int d[V]; memset(d, 0x3F, sizeof(d)); d[T] = 0;
    static deque<int> Q; Q.push_back(T);
    while(Q.size())
    {
        int dt, no = Q.front(); Q.pop_front();
        for(etype *i = e[no]; i; i = i->next)
            if(i->pair->u && (dt = d[no] - i->c) < d[i->t])
                (d[i->t] = dt) <= d[Q.size() ? Q.front() : 0]
                    ? Q.push_front(i->t) : Q.push_back(i->t);
    }
    for(int i = 0; i < n; ++i)
        for(etype *j = e[i]; j; j = j->next)
            j->c += d[j->t] - d[i];
    piS += d[S];
    return d[S] < maxint;
}

int ab[V], *pab[V], w[V];

struct lt
{
    bool operator()(int* p1,int* p2) {return *p1 < *p2;}
};

int main()
{
    int t;
    scanf("%d",&t);
    while(t--)
    {
        memset(e,0,sizeof(e));
        Pe = Te;
        static int m, k;
        scanf("%d %d", &m, &k);
        int abz = 0;
        for(int i = 0; i < m; ++i)
        {
            scanf("%d", pab[abz] = &ab[abz]), abz++;
            scanf("%d", pab[abz] = &ab[abz]), abz++;
            scanf("%d", &w[i]);
        }
    }
}

```



```

    }
    sort(&pab[0], &pab[abz], lt());
    int c=0xDEADBEEF; n=0;
    for(int i = 0; i < abz; ++i)
    {
        if(c != *pab[i]) c = *pab[i], ++n;
        *pab[i] = n;
    }
    ++n, S = 0, T = n++;
    for(int i = 0; i < T; ++i) addedge(i, i+1, 0, k);
    for(int i = 0; i < m; ++i) addedge(ab[i+1], ab[i+i+1], -w[i], 1);
    piS = cost = 0;
    while(modlabel())
        do memset(v, 0, sizeof(v));
        while(aug(S, maxint));
    printf("%d\n", -cost);
}
return 0;
}

```

Bài viết này đã được chỉnh sửa 7 lần. Lần chỉnh sửa cuối bởi zkw, ngày 7 tháng 7 năm 2011, 2:16 sáng.